PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Naskah Soal Praktikum 3

Transaction & Concurrency Control

Jumat, 13 November 2026

Oleh:
13524089 - Aurelia Jennifer Gunawan
13524116 - Eliana Natalie Widjojo

IF3140 - Sistem Basis Data

2026

Petunjuk

1. Kerjakan setiap soal praktikum ini dengan baik
2. Diperkenankan untuk menggunakan dokumentasi PostgreSQL resmi
3. Nama database: **airport**
4. **Dilarang Menyontek**
5. **docker exec praktikum-db-1 pg_dump -U postgres -d airport -h localhost -p 5432 > [DATABASE_FILE_EXPORTED].sql**

airport


passengerdetails
passenger_id
birthdate
sex
street
city
zip
country
emailaddress
telephoneno
passenger
passenger_id
passportno
firstname
lastname
airport_geo
airport_id
name
city
country
latitude
longitude
geolocation
booking
booking_id
flight_id
seat
passenger_id
price
flight
flight_id
flightno
from
to
departure
arrival
airline_id
airplane_id
airplane
airplane_id
capacity
type_id
airline_id
airplane_type
type_id
identifier
description
airport
airport_id
iata
icao
name
employee
employee_id
firstname
lastname
birthdate
sex
street
city
zip
country
emailaddress
telephoneno
salary
department
username
password
airline
airline_id
iata
airlinename
base_airport
weatherdata
log_date
time
station
temp
humidity
airpressure
wind
weather
winddirection
flightschedule
flightno
from
to
departure
arrival
airline_id
monday
tuesday
wednesday
thursday
friday
saturday
sunday

1. Tiket Setengah Jadi

Yeho ditugaskan untuk membantu bagian ticketing maskapai. Setiap ada penumpang baru yang memesan tiket, Yeho harus melakukan tiga hal: mendaftarkan penumpang, mengisi detail datanya, lalu mencatat booking-nya. Suatu hari, sistem tiba-tiba error di tengah proses. Penumpangnya sudah terdaftar, detailnya sudah terisi, tetapi booking-nya tidak pernah tersimpan. Akibatnya, ada "penumpang hantu" yang punya data lengkap tetapi tidak punya tiket.

Zoho tidak mau kejadian ini terulang. Ia meminta Yeho memperlakukan ketiga langkah tersebut sebagai **satu kesatuan transaksi:**

1. Data penumpang disimpan ke dalam tabel *passenger*
2. Detail penumpang disimpan ke dalam tabel *passengerdetails*
3. Data pemesanan tiket disimpan ke dalam tabel *booking*

Jika satu langkah saja gagal, seluruh transaksi harus dibatalkan sehingga tidak ada data yang tersimpan setengah jadi.

Bagian I

Lakukan sebuah transaksi (melibatkan 3 langkah di atas) menggunakan data:

- passenger_id = 7001
- passportno = 'B1234567'
- firstname = 'Rina'
- lastname = 'Kartika'
- birthdate = '2003-08-17'
- sex = 'F'
- city = 'Bandung'
- country = 'Indonesia'
- booking_id = 72001
- flight_id = 651
- seat = '14A'
- price = 250.00

Bagian II

Lakukan sebuah transaksi (melibatkan 3 langkah di atas) menggunakan data:

- passenger_id = 7002
- passportno = ‘C7654321’
- firstname = ‘Dimas’
- lastname = ‘Haryanto’
- birthdate = ‘2002-03-21’
- sex = ‘M’
- city = ‘Bandung’
- country = ‘Indonesia’
- booking_id = 72002
- flight_id = 9999
- seat = ‘14B’
- price = 250.00

Bagian III

A. Apa yang dimaksud dengan *atomicity*?
B. Apa fungsi dari COMMIT dan ROLLBACK?
C. Apa yang terjadi jika ada salah satu proses yang gagal di dalam suatu transaksi?

HINT: Gunakan START TRANSACTION, COMMIT, dan ROLLBACK dalam menjalankan sebuah transaksi

Jawaban:

Query Bagian I	

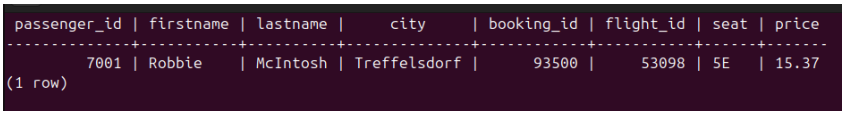
```
START TRANSACTION;

UPDATE passenger
SET firstname = 'Rina', lastname = 'Kartika', passportno
= 'B1234567'
WHERE passenger_id = 7001;

UPDATE passengerdetails
SET birthdate = '2003-08-17', sex = 'F', city =
'Bandung', country = 'Indonesia'
WHERE passenger_id = 7001;

INSERT INTO booking (booking_id, flight_id, seat, price,
passenger_id)
VALUES (72001, 651, '14A', 250.00,7001);

COMMIT;
```

SS Bagian I	
Query Verifikasi Bagian I (diberikan)	SELECT p.passenger_id, p.firstname, p.lastname, pd.city, b.booking_id, b.flight_id, b.seat, b.price FROM passenger p JOIN passengerdetails pd ON pd.passenger_id = p.passenger_id JOIN booking b ON b.passenger_id = p.passenger_id WHERE p.passenger_id = 7001;
SS Verifikasi Bagian I	
Query Bagian II	START TRANSACTION; UPDATE passenger SET firstname = 'Dimas', lastname = 'Haryanto', passportno = 'C7654321' WHERE passenger_id = 7001; UPDATE passengerdetails SET birthdate = '2002-03-21', sex = 'M', city = 'Bandung', country = 'Indonesia' WHERE passenger_id = 7001; INSERT INTO booking (booking_id, flight_id, seat, price, passenger_id) VALUES (72002, 9999, '14B', 250.00, 7001); COMMIT;

SS Bagian II	
Query Verifikasi Bagian II (diberikan)	SELECT * FROM passenger WHERE passenger_id = 7002; SELECT * FROM passengerdetails WHERE passenger_id = 7002; SELECT * FROM booking WHERE booking_id = 72002;
SS Verifikasi Bagian II	
Jawaban Bagian III.A	Atomicity adalah transaksi dilakukan secara utuh (all-or-nothing). Jika terdapat error di tengah, semuanya di abort/rollback ke awal. Tidak boleh ada transaksi yang hanya berjalan setengah-setengah.
Jawaban Bagian III.B	Commit berfungsi untuk mencatat transaksi ke dalam database agar dapat terlihat oleh user lain. Rollback berfungsi untuk mengembalikan kondisi database ke semula, sebelum transaksi.
Jawaban Bagian III.C	JIka salah satu proses gagal di dalam suatu transaksi, semua proses yang sudah dijalankan di-abort/rollback. Proses yang sudah dijalankan dibatalkan dan database kembali seperti semula.

2. Audit Tiket Penerbangan

Bagian administrasi maskapai AirRush sedang melakukan audit harga dan jumlah pemesanan tiket pada penerbangan dengan flight_id = 5. Salah satu tiket yang sedang diperiksa adalah tiket dengan booking_id = 34.

Pada saat audit berlangsung, staf administrasi sedang memeriksa harga tiket dan jumlah pemesanan yang terdaftar. Di waktu yang bersamaan, staf reservasi melakukan perubahan harga tiket dan menambahkan pemesanan baru pada penerbangan yang sama.

Untuk menghindari perubahan data yang tidak konsisten selama proses audit, sistem administrasi menggunakan isolation level REPEATABLE READ, sedangkan sistem reservasi menggunakan isolation level default PostgreSQL, yaitu READ COMMITTED. Berikut merupakan urutan transaksi yang dilakukan oleh kedua staf:

Waktu	Transaksi A (REPEATABLE READ)	Transaksi B (READ COMMITTED)
T1	`BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;`	
T2	`SELECT price FROM booking WHERE booking_id = 34;`	
T3	`SELECT COUNT(*) FROM booking WHERE flight_id = 5;`	

T4		`BEGIN;`
		
		`UPDATE booking SET price = 1800.00 WHERE booking_id = 34;`
		
		`INSERT INTO booking (booking_id, flight_id, seat, passenger_id, price) VALUES (90001, 5, '99Z', 4, 500.00);`
		
		`COMMIT;`

T5	`SELECT price FROM booking WHERE booking_id = 34;`	
T6	`SELECT COUNT(*) FROM booking WHERE flight_id = 5;`	
T7	`UPDATE booking SET price`	

	`= price - 100 WHERE booking_id = 34;`	
T8	`COMMIT`	

a. Berapakah nilai price yang ditampilkan pada Transaksi A di waktu T5? Jelaskan alasan mengapa nilai tersebut bisa muncul.

130.23 karena Transaksi A adalah repeatable read. Value yang transaksi A baca adalah value waktu ia BEGIN. Transaksi A tidak dapat membaca value yang sudah di-commit oleh transaksi lain sewaktu transaksi A masih berjalan.

b. Berapakah hasil COUNT(*) pada Transaksi A di waktu T6? Berdasarkan aturan PostgreSQL, apakah terjadi Phantom Read? Jelaskan alasannya.

0. Tidak terjadi phantom read karena transaksi B menambahkan satu baris, tapi hasil COUNT(*) tetap 0. Ini terjadi karena transaksi A adalah repeatable read. Pada PostgreSQL, repeatable read tidak mengizinkan phantom read.

c. Apa yang terjadi (berhasil/gagal/error) ketika Transaksi A menjalankan perintah `UPDATE` pada waktu T7? Jelaskan penyebabnya.

```
airport=*# UPDATE booking SET price = price - 100 WHERE booking_id = 34;

ERROR:  could not serialize access due to concurrent update
```

Terjadi error: could not serialize access due to concurrent update.

Penyebab error adalah karena transaksi A berusaha meng-update value yang sudah di-update transaksi B. Oleh karena itu, terjadi error karena transaksi tidak dapat di-serialisasi karena concurrent update.

3. Asal Basdat Senang

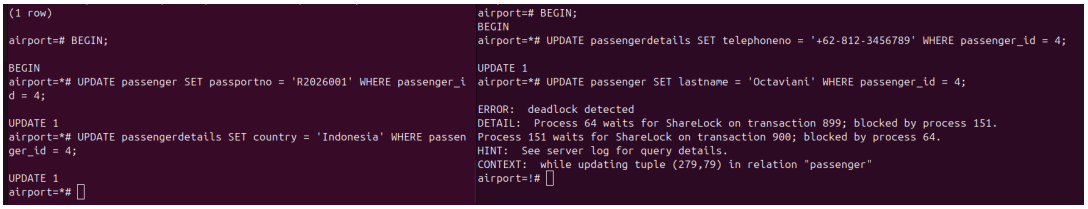
Kementerian Basdat menginstruksikan pemblokiran darurat pada penerbangan AM5199 (`flight_id = 5`) karena pesawat tersebut akan disita sementara untuk evakuasi khusus Pejabat Basdat. Staf kementerian menggunakan sistem internal untuk mengubah status penerbangan dan membatalkan tiket terkait. Ternyata, di saat yang bersamaan, sistem reservasi publik sedang memproses pembayaran *flash sale* dari pelanggan untuk tiket pada penerbangan yang sama. Akibatnya, di *database* terjadi dua transaksi pada waktu berbeda yang menjalankan perintah seperti *schedule* di bawah ini.

Timestep	Transaction T1 - Kementerian Basdat	Transaction T2 - Sistem Reservasi
1	`BEGIN;`	
2		`BEGIN;`
3	`UPDATE passenger SET passportno = 'R2026001' WHERE passenger_id = 4;`	
4		`UPDATE passengerdetails SET telephoneno = '+62-812-3456789' WHERE passenger_id = 4;`
5	`UPDATE passengerdetails SET country = 'Indonesia' WHERE passenger_id = 4;`	
6		`UPDATE passenger SET lastname = 'Octaviani' WHERE passenger_id = 4;`

Coba jalankan kedua transaksi tersebut pada masing-masing satu buah terminal (untuk mensimulasikan sistem kementerian dan sistem reservasi yang masing-masing mengirimkan perintah ke database) dengan urutan penulisan perintah sama persis dengan *schedule* di atas. *Screenshot* hasilnya!

1. Jelaskan kenapa *deadlock* bisa terjadi dengan menjelaskan jenis-jenis *lock* dan *resource* yang di-*lock* sesuai *schedule* di atas!

2. Jelaskan kenapa *deadlock* bisa terjadi dengan menjelaskan jenis-jenis *lock* dan *resource* yang di-*lock* sesuai *schedule* di atas!

SS Poin 1


Jawaban Poin 2
Deadlock terjadi karena: 1. T1 mengunci passenger dengan exclusive lock 2. T2 mengunci passengerdetails dengan exclusive lock 3. T1 menunggu kunci passenger details dibuka (kunci masih dipegang T2) 4. T2 menunggu kunci passenger dibuka (kunci masih dipegang T1) Akhirnya, T1 dan T2 saling menunggu satu sama lain.

4. Slot Terakhir

Kapasitas penerbangan **flight_id = 706** dibatasi maksimal 24 tiket. Saat ini sudah ada 23 tiket terdaftar.

Untuk mencegah kelebihan kapasitas (overbooking), sistem tiket menjalankan prosedur berikut setiap kali ada permintaan reservasi baru:

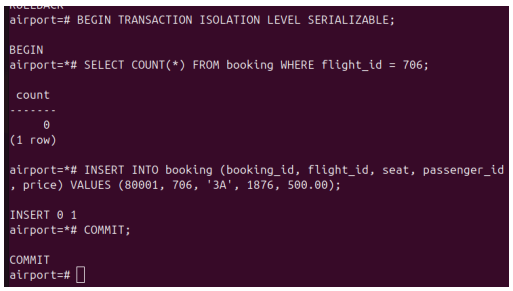
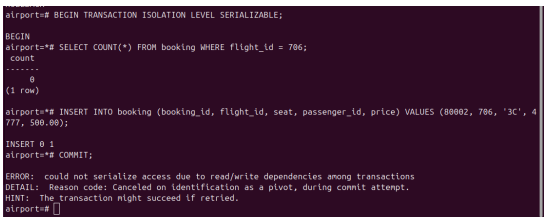
1. Memeriksa sisa kuota dengan menjalankan SELECT COUNT(*) FROM booking WHERE flight_id = 706;.
2. Jika jumlah tiket saat ini masih kurang dari 24 (COUNT(*) < 24), sistem akan memasukkan data tiket baru ke tabel booking.
3. Jika kuota sudah mencapai 24, pemesanan ditolak.

Pada satu waktu, dua agen tiket (Transaksi T1 dan Transaksi T2) menerima pesanan pelanggan secara bersamaan dan menjalankan prosedur tersebut pada level isolasi SERIALIZABLE.

Timestep	T1	T2
1	BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;	
2		BEGIN TRANSACTION ISOLATION LEVEL

		SERIALIZABLE;
3	SELECT COUNT(*) FROM booking WHERE flight_id = 706;	
4		SELECT COUNT(*) FROM booking WHERE flight_id = 706;
5	INSERT INTO booking (booking_id, flight_id, seat, passenger_id, price) VALUES (80001, 706, '3A', 1876, 500.00);	
6		INSERT INTO booking (booking_id, flight_id, seat, passenger_id, price) VALUES (80002, 706, '3C', 4777, 500.00);
7	COMMIT;	
8		COMMIT;

1. Jalankan schedule di atas menggunakan dua terminal psql terpisah sesuai urutan timestep

SS T1	SS T2
	

2. Jelaskan mengapa transaksi digagalkan pada level SERIALIZABLE meski T1 dan T2 meng-insert baris yang berbeda

Jawaban No. 2	Transaksi digagalkan pada level SERIALIZABLE karena PostgreSQL menggunakan SSI (Serializable Snapshot Isolation) yang mengecek keterkaitan read-write (read/write dependency).

3. Apa yang terjadi jika menggunakan isolation level seperti read commited? Kenapa?

Jawaban No. 3	Jika menggunakan READ COMMITTED, setiap [erintah hanya melihat data yang sudah di-commit. Sehingga flight yang di-booking ada 25 tiket karena dilakukan 2 kali

5. My Booking Gueh (MBG)

Dua petugas AirRush, Nina dan Nino, sedang memproses biaya tambahan untuk pemesanan yang sama. Nina menambahkan biaya bagasi sebesar 10.00, sedangkan Nino menambahkan biaya pemilihan kursi sebesar 20.00. Kedua biaya harus tercatat pada harga akhir pemesanan.

Aplikasi loket membaca booking.price, menghitung harga baru dari nilai yang dibaca, lalu menyimpan hasil perhitungannya ke baris yang sama. Setiap petugas mengonfirmasi perubahan setelah transaksinya berhasil di-commit.

Gunakan dua terminal psql yang terhubung ke database airport milik kelompokmu. Percobaan menggunakan **booking_id = 1** dengan harga awal 105.09. Sebelum mulai, periksa harga awal di salah satu terminal.

```
SELECT booking_id, price
FROM booking
WHERE booking_id = 1;
```

A. Analisis Transaksi Awal

Analisis urutan transaksi berikut. Bagian ini cukup dijawab dari tabel. Pada T5 dan T7, aplikasi menyimpan angka hasil perhitungan dari pembacaan masing-masing.

Langkah	Terminal A (Nina)	Terminal B (Nino)
T1	BEGIN ISOLATION LEVEL READ COMMITTED;	
T2	SELECT price FROM booking WHERE booking_id = 1;	
T3		BEGIN ISOLATION LEVEL READ COMMITTED;
T4		SELECT price FROM booking WHERE booking_id = 1;

T5	UPDATE booking SET price = 105.09 + 10.00 WHERE booking_id = 1;	
T6	COMMIT;	
T7		UPDATE booking SET price = 105.09 + 20.00 WHERE booking_id = 1;
T8		COMMIT;
T9	SELECT price FROM booking WHERE booking_id = 1;	
T10		SELECT price FROM booking WHERE booking_id = 1;

1. Berapa harga yang dibaca Nina dan Nino? Berapa harga akhir yang tersimpan setelah kedua transaksi selesai?
2. Biaya mana yang hilang dari harga akhir? Jelaskan mengapa kedua transaksi tetap bisa berhasil melakukan COMMIT.
3. Sertakan screenshot

B. Perbaikan dengan row-level lock

Tulis dan jalankan SQL di kedua terminal. Mulai dari harga awal 105.09 dan tetap gunakan isolasi READ COMMITTED serta alur baca harga -> hitung harga baru -> simpan -> COMMIT.

1. Di terminal A, mulai transaksi Nina. Baca harga pemesanan sambil mengambil row-level lock dan pertahankan lock hingga transaksi berakhir.
2. Di terminal B, mulai transaksi Nino dan jalankan pembacaan dengan lock pada pemesanan yang sama **sebelum Nina melakukan COMMIT**. Amati apa yang terjadi di terminal B.
3. Kembali ke terminal A. Simpan harga baru Nina, lalu lakukan COMMIT. Catat harga yang dikembalikan oleh pembacaan Nino setelah itu.
4. Gunakan harga yang baru dibaca Nino untuk menghitung dan menyimpan harga barunya, lalu lakukan COMMIT. Tampilkan harga akhir pemesanan.

Isi tabel urutan SQL dan sertakan screenshot yang menunjukkan terminal B saat menunggu serta hasil pembacaan Nino dan harga akhir. Setelah bukti dicatat, kembalikan harga awal.

```
UPDATE booking
SET price = 105.09
WHERE booking_id = 1;
```

C. Penjelasan

Saat Nina masih memegang lock **pada booking_id = 1,** tentukan apakah masing-masing operasi berikut harus menunggu jika dijalankan di sesi lain. Asumsikan tidak ada lock lain yang menghalangi kedua operasi tersebut.

1. Membaca price pada booking_id = 1 dengan SELECT biasa.
2. Menjalankan UPDATE harga pada satu baris booking dengan booking_id yang berbeda.

Jelaskan alasan untuk masing-masing operasi dan kaitkan dengan keuntungan row-level lock dibandingkan menjalankan seluruh transaksi pemesanan satu per satu.

Jawaban:

Hasil dan Analisis Bagian - A	Terminal A (Nina):
	`airport=# BEGIN ISOLATION LEVEL READ COMMITTED;`
	
	`BEGIN`
	`airport=*# SELECT price FROM booking WHERE booking_id = 1;`
	
	` price`
	`--------`
	` 110.10`
	`(1 row)`
	
	`airport=*# UPDATE booking SET price = 110.10 + 10.00 WHERE booking_id = 1`
	`;`
	
	`UPDATE 1`
	`airport=*# COMMIT;`
	
	`COMMIT`
	`airport=# SELECT price FROM booking WHERE booking_id = 1;`
	
	` price`
	`--------`
	` 130.10`
	`(1 row)`
	
	`airport=#`
	
	Terminal Nino:

```
airport=# BEGIN ISOLATION LEVEL READ COMMITTED;

BEGIN
airport=*# SELECT price FROM booking WHERE booking_id = 1;

 price
--------
 110.10
(1 row)

airport=*# UPDATE booking SET price = 110.10 + 20.00 WHERE booking_id = 1;

UPDATE 1
airport=*# COMMIT;

COMMIT
airport=# SELECT price FROM booking WHERE booking_id = 1;

 price
--------
 130.10
(1 row)

airport=#
```

Berapa harga yang dibaca Nina dan Nino? Berapa harga akhir yang tersimpan setelah kedua transaksi selesai?

Pada mulanya:
Nina -> 110.10
Nino -> 110.10

Pada akhirnya setelah kedua transaksi selesai:
Nina -> 130.10
Nino -> 130.10

Biaya yang ditulis pada transaksi di terminal Nina hilang dari harga akhir (terjadi write skew). Keduanya masih bisa commit karena isolation level adalah read committed. Jadi, Nina update harga, kemudian Nino menimpa harga. Akhirnya, biaya yang ditulis di terminal Nina hilang. Read commited berarti dapat membaca value yang sudah di-commit oleh transaksi lain.

Urutan SQL di kedua terminal - B

Langkah	Terminal A (Nina)	Terminal B (Nino)
T1	BEGIN ISOLATION LEVEL READ COMMITTED;	
T2	SELECT price	

	FROM booking WHERE booking_id = 1 FOR UPDATE;	
T3		BEGIN ISOLATION LEVEL READ COMMITTED;
T4		SELECT price FROM booking WHERE booking_id = 1 FOR UPDATE;
T5	UPDATE booking SET price = 105.09 + 10.00 WHERE booking_id = 1;	
T6	COMMIT;	
T7		UPDATE booking SET price = 105.09 + 20.00 WHERE booking_id = 11;
T8		COMMIT;
T9	SELECT price FROM booking WHERE booking_id = 1;	
T10		SELECT price FROM booking WHERE

	<table><tr><td></td><td></td><td>booking_id = 1;</td></tr></table>

Screenshot dan hasil akhir - B	**Terminal Nina:**
	`airport=# BEGIN ISOLATION LEVEL READ COMMITTED;`
	
	`BEGIN`
	`airport=*# SELECT price FROM booking WHERE booking_id = 1 FOR UPDATE;`
	
	` price`
	`--------`
	` 105.09`
	`(1 row)`
	
	`airport=*# UPDATE booking SET price = 105.09 + 10.00 WHERE booking_id = 1`
	`;`
	`UPDATE 1`
	`airport=*# COMMIT;`
	
	`COMMIT`
	`airport=# SELECT price FROM booking WHERE booking_id = 1;`
	
	` price`
	`--------`
	` 115.09`
	`(1 row)`
	
	`airport=#`
	
	Terminal Nino:
	`airport=# BEGIN ISOLATION LEVEL READ COMMITTED;`
	
	`BEGIN`
	`airport=*# SELECT price FROM booking WHERE booking_id = 1 FOR UPDATE;`
	` price`
	`--------`
	` 115.09`
	`(1 row)`
	
	`airport=*# UPDATE booking SET price = 105.09 + 20.00 WHERE booking_id = 11;`
	
	`UPDATE 1`
	`airport=*# COMMIT;`
	
	`COMMIT`
	`airport=# SELECT price FROM booking WHERE booking_id = 1;`
	
	` price`
	`--------`
	` 115.09`
	`(1 row)`
	
	`airport=#`
	
	Pada mulanya sebelum update di transaksi dilakukan:
	Nina -> 105.09
	Nino -> 115.09
	
	Pada akhirnya setelah kedua transaksi selesai:
	Nina -> 115.09
	Nino -> 115.09

Penjelasan - C	Operasi membaca price pada booking_id = 1 dengan SELECT biasa masih menunggu karena lock yang digunakan adalah exclusive lock, sehingga baik operasi read maupun write tetap harus menunggu. Sedangkan operasi untuk menjalankan UPDATE harga pada satu baris booking dengan booking_id yang berbeda masih bisa dilakukan karena locking dilakukan pada level row/tuple sehingga tuple denagn booking_id yang berbeda tidak terpengaruhi lock. Keuntungan dari low-level locking dapat dilihat disini, yaitu tidak mungkin ada konflik pada row yang sama, dan memberikan fleksibilitas dibandingkan table level locking yang mengunci keseluruhan tabel. Dengan row-level locking, DBMS masih bisa menjalankan transaksi pada row yang berbeda.

6. Anomali 67

Tuan Wong dan Tuan Nut merupakan dua staf bagian reservasi pada sebuah maskapai penerbangan. Mereka bertugas melakukan penyesuaian harga tiket untuk pelanggan.

Suatu hari, Tuan Wong diminta memberikan diskon sebesar $100 kepada seorang penumpang, sementara Tuan Nut diminta menambahkan biaya layanan sebesar $200 pada pemesanan yang sama.

Harga awal tiket adalah $1730.64. Kedua staf terlebih dahulu membaca harga tiket sebelum melakukan perubahan. Namun, karena tidak ada koordinasi, kedua transaksi tersebut berjalan secara bersamaan dengan urutan sebagai berikut.

Langkah	Tuan Wong	Tuan Nut
T1	BEGIN;	BEGIN;
T2	SELECT price FROM booking WHERE booking_id = 67;	
T3		SELECT price FROM booking WHERE booking_id = 67;

T4	UPDATE booking SET price = 1630.64 WHERE booking_id = 67;	
T5		UPDATE booking SET price = 1930.64 WHERE booking_id = 67;
T6	COMMIT;	
T7		COMMIT;

1. Jalankan skema query tersebut menggunakan dua tab terminal PostgreSQL. Apa yang terjadi pada T5? Berapakah harga akhir tiket setelah kedua transaksi selesai? Jelaskan mengapa hal tersebut terjadi!
2. Identifikasi permasalahan concurrency yang terjadi pada skenario di atas. Mengapa penggunaan transaksi dan mekanisme locking PostgreSQL belum cukup untuk menjamin hasil perhitungan harga yang benar?

Jawaban:

Jawaban Bagian 1	Pada T5, sebelum transaksi tuan Wong commit, T5 harus menunggu tuan wong commit. Harga akhir tiket adalah 1930.64 setelah kedua transaksi selesai. Hal ini terjadi karena Tuan Wong dan Tuan Nut memodifikasi row/tuple yang sama, yaitu tuple dengan booking_id = 67, sehingga ketika Tuan Wong commit dia melepaskan lock pada row dengan booking_id 67 dan Tuan Nut bisa update.
Query Verifikasi (Diberikan)	SELECT price FROM booking WHERE booking_id = 67;

SS *Query* Bagian 1	Tuan Wong:
	`airport=# BEGIN;`
	`BEGIN`
	`airport=*# SELECT price FROM booking WHERE booking_id = 67;`
	` price`
	`--------`
	` 197.80`
	`(1 row)`
	
	`airport=*# UPDATE booking SET price = 1930.64 WHERE booking_id = 67;`
	`UPDATE 1`
	`airport=*# COMMIT;`
	`COMMIT`
	`airport=#`
	
	`airport=# SELECT price FROM booking WHERE booking_id = 67;`
	`  price`
	`---------`
	` 1930.64`
	`(1 row)`
	
	Tuan Nut:
	`airport=# BEGIN;`
	`BEGIN`
	`airport=*# SELECT price FROM booking WHERE booking_id = 67;`
	` price`
	`--------`
	` 197.80`
	`(1 row)`
	
	`airport=*# UPDATE booking SET price = 1630.64 WHERE booking_id = 67;`
	`UPDATE 1`
	`airport=*# COMMIT;`
	`COMMIT`
	`airport=#`
	
	`airport=# SELECT price FROM booking WHERE booking_id = 67;`
	`  price`
	`---------`
	` 1930.64`
	`(1 row)`
	
	`airport=#`

Jawaban Bagian 2	Penggunaan transaksi dan mekanisme locking masih belum cukup karena Tuan Wong membaca himpunan baris yang diubah oleh Tuan Nut sedangkan Tuan Nut juga membaca

	himpunan baris yang diubah oleh Tuan Wong.

Pembagian Kerja

NIM	Tugas
13524089	Mengerjakan soal SQL, membantu yang teori
13524116	Mengerjakan soal teori, membantu query SQL

